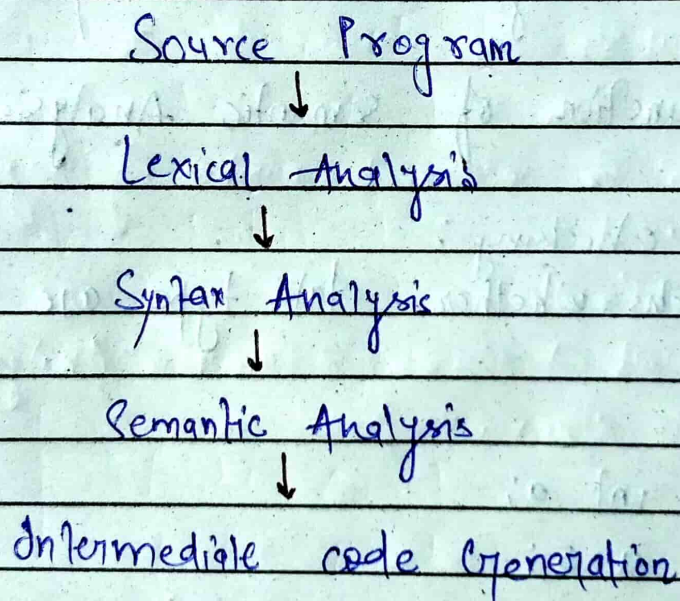


Phase 3: Semantic Analysis :-

Semantic Analysis is the third phase of a compiler, performed after lexical analysis and Syntax Analysis.

Its main purpose is to check whether a program is meaningful and logically correct according to the rules of the programming language.

→ Compiler Flow:



→ What does semantic Analysis do?

Syntax Analysis checks the structure of a program while semantic analysis checks its meaning.

eg:

```
int a;  
a = 10;
```

This is syntactically and semantically correct.

But :

```
int a;  
a = "Hello";
```

The statement may have a valid syntax, but it has a type mismatch because a is an integer and "hello" is a string.

Semantic analysis detects such problems.

→ Main function of semantic analysis :

1. Type checking :
checks whether data type are compatible.

Eg ;

```
int a;  
a = 5;
```

// correct ✓

```
int a;  
a = 'hello';
```

// Type error

2. Identifier Checking:

Checks whether variable and functions are properly declared.

Eg:

```
int a;  
a = 10;  
// correct
```

but:

```
b = 10;
```

If b has not been declared, it produces a semantic error.

3. Scope checking:

Checks whether an identifier is being used within its valid scope.

Eg:

A local variable declared inside a function may not be accessible outside that function.

4. Declaration Checking:

Check whether identifiers are declared correctly before their uses, according to the language rules.

5. Function/Parameter checking:
checks whether the number and type of argument match the function declaration.

Eg:

sum(int, int)

Calling:

sum(10, 20)

is valid

But:

sum(10)

has an incorrect number of arguments

→ Semantic Errors:

Error detected during semantic Analysis is called semantic errors.

Eg:

int x;

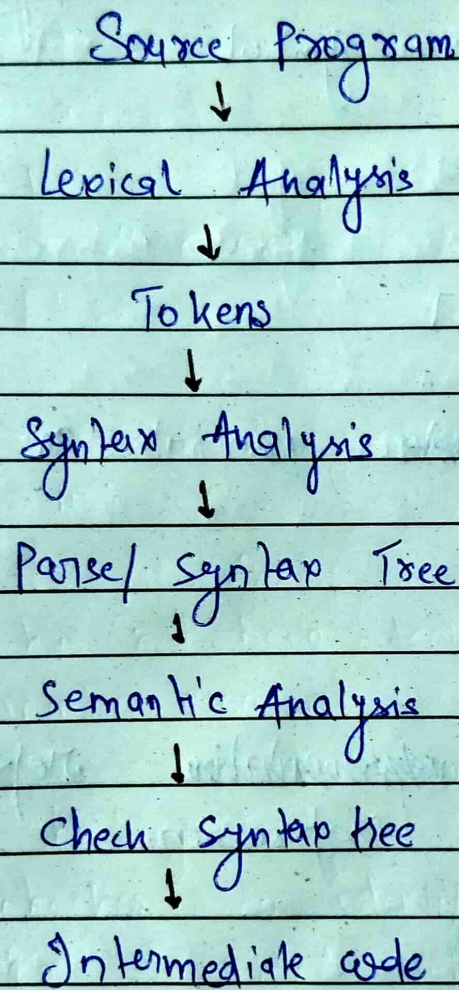
x = "Hello"; → Type mismatch

y = 10; → undeclared identifier

sum(10); → wrong number of arguments

These statement can have correct syntax, but their meaning is invalid.

→ Role of Syntax / Parse tree: ~
Semantic analysis works mainly on the syntax tree/
Parse tree produced by syntax analysis.



During semantic analysis, information such as type, value, scope, and identifies information can be attached to nodes of the syntax tree.

This leads directly to the concept of ~~attributes & abstract~~ **attributes & abstract grammar**, which are the first specific topic.

→ Attribute Grammars :-

→ Attribute :

An attribute is additional information associated with a grammar symbol.

for eg:

$$E \rightarrow E + T$$

we can ~~consider~~ associate an attribute such as:

$E.val$

$T.val$

where, $.val$ ~~representation~~ represents the value of the expression.

for

$$3 + 5$$

we can calculate :

$$E.val = E_1.val + T.val$$

so:

$$E.val = 3 + 5 = 8$$

The attribute provides semantic information about the grammar symbol.

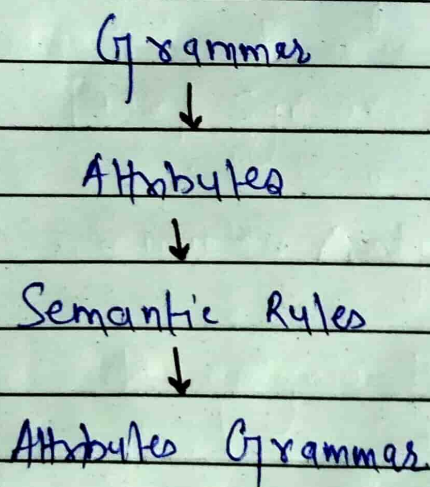
→ Attribute Grammar:

An attribute Grammar is a CFG in which grammar symbols are associated with attributes and productions have semantic rules for calculate those attributes.

In simple words:

Attribute Grammar = CFG + Attributes + Semantic Rules

→ Basic Structure:



Eg:

$E \rightarrow E_1 + T$

Suppose every expression has an attribute:

$E.val$

$T.val$

We can define the semantic rule:

$$E.val = E_1.val + T.val$$

here:

- $E.val \Rightarrow$ attribute of E
- $E_1.val \Rightarrow$ value of first expression
- $T.val \Rightarrow$ value of term
- $+$ $\Rightarrow$ operation

for

$$3 + 5$$

we get:

$$E_1.val = 3$$

$$T.val = 5$$

$$E.val = E_1.val + T.val$$

$$= 3 + 5$$

$$= 8$$

→ Attribute Grammar Example with parse Tree:

Consider:

$$E \rightarrow E_1 + T$$

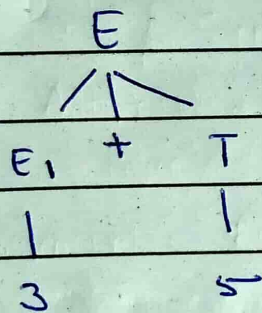
$$E \rightarrow T$$

$$T \rightarrow \text{num}$$

input :

3+5

Parse Tree:



→ Why do we need Attribute Grammar?

A CFG tells us whether the structure is valid.

Attribute Grammar allows us to attach meaning/ information to that structure.

Eg:

Syntax
↓

$E \rightarrow E + T$

$T \rightarrow \text{num}$

Semantic Information
↓

$E. \text{val}$

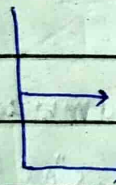
$T. \text{val}$

It can be used for:

- calculating expression values
- type checking
- generating intermediate code
- storing information about identifiers
- propagating information through a syntax tree.

→ Types of Attributes :-

Attributes



Synthesized Attributes
Inherited Attributes

(i) Synthesized Attributes:

A synthesized attribute is an attribute whose value is calculated using the attribute of its child nodes in a syntax tree.

In simple words:

Information flows from **child** to **parent** (bottom to up).

Eg:

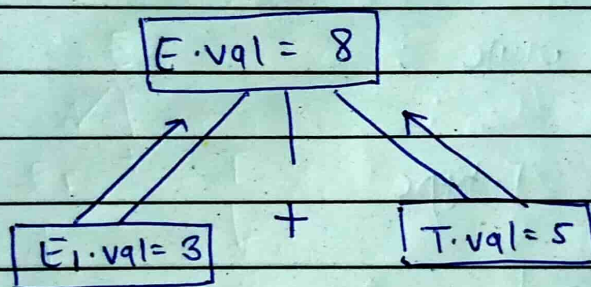
$$3 + 5$$

Production:

$$E \rightarrow E_1 + T$$

Semantic rule:

$$E.val = E_1.val + T.val$$



Information flows upward ↑

vi) Inherited Attributes:

An inherited attribute is an attribute whose value is calculated using information from its parents or sibling nodes in a syntax tree.

In simple words:

Information flows from **parent** to **child** or from **sibling** to **sibling**.

Eg: Passing a data type

Consider the declaration:

int a, b;

Grammar:

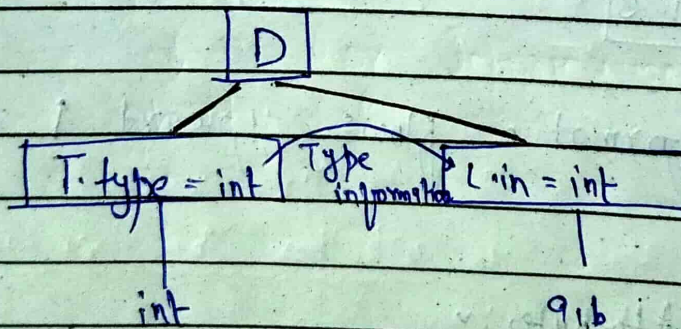
$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id}, \text{id}$

Semantic rule:

$L.in = T.type$



Information passes from T to L

→ Evaluation & flow of attribute in a syntax tree:

→ Attribute evaluation!

Attribute evaluation means calculating the values of all the attribute attached to grammar symbols in a syntax tree using their semantic rules.

Eg:

$$E \rightarrow E_1 + T$$

with the semantic rule:

$$E.val = E_1.val + T.val$$

$$\text{if } E_1.val = 3$$

$$T.val = 5$$

then:

$$E.val = 3 + 5 = 8$$

Attribute evaluation:

The process of determining the values of attributes of nodes in a syntax tree

→ Flow of attributes:

Flow of attributes means the direction in which attribute information move b/w nodes of a syntax tree.

There are two types:

(i) Synthesized flow

Child



Parent

Information moves upward

(ii) Inherited flow:

Parent



child

or Information may flow b/w siblings.

→ Symbol Table :-

A symbol table is an important data structure used by the compiler to store information about **identifiers** used in a program.

Identifiers include :

- variable
- function
- constant
- Arrays
- classes
- Parameters
- Other user-defined names

In simple words :

Symbol table is like a record maintained by the compiler that stores information about every important name in the program.

→ Why is a Symbol Table needed?

Consider :

```
int age;  
float salary;  
age = 20;  
salary = 40,000.50;
```

Page No. _____
Date ____/____/____

The compiler need to ~~member~~ remember :

age $\rightarrow$ Int
salary $\rightarrow$ Float

So when it see :
age = 20;

it can check whether age exists and whether 20 is compatible with its type.

$\therefore$ the symbol table helps the compiler perform :

- Declaration checking
- Type checking
- Scope checking
- Function checking
- Storage allocation
- Code generation

→ Basic Structure of Symbol table:

A symbol table can be represented as a table:

Name	type	scope	Size	Address	other inf ⁿ
age	int	local	4	1000	variable
salary	float	local	4	2000	variable
sum	function	global	-	8000	function

→ Symbol Attribute:

Symbol Attribute are the pieces of information stored about an identifier.

for eg:
int x;

The symbol table may store:

Name : x
Type : int
Kind : variable
scope : local/global
size : 4 bytes
Address : some memory location

→ Symbol table management:

Symbol table management means performing operation such as:

Insert

Search

Update

Delete

Scope Handling

→ Insert Operation:

When a new identifier is declared, the compiler inserts it into the symbol table.

Eg:

int x;

The compiler creates:

x → int

and inserts it.

→ lookup / search operation:
When an identifier is used, the compiler searches for it.

Eg:

$x = 10;$

Compiler asks:
Is x present?

If $x \rightarrow \text{int}$
exist, the identifier is valid.

If x is not present, the compiler may report:
undeclared identifier: x

→ Update Operation:

Sometimes information associated with an identifier needs to be updated.

for eg: during compilation, the compiler may initially know:

$x \rightarrow \text{variable}$

and later associated additional information.

$x \rightarrow \text{int, variable, local, offset} = 8$

→ Delete Operation:

When a scope ends, symbols belonging only to that scope may be removed from the active symbol table.

Eg:

```
{
    int x;
}
```

x is valid inside the block.

After leaving the block, x is no longer visible.

→ Handling Scope in

~~Handling Scope in~~

Consider:

```
int x;
{
    int y;
    x = 10;
    y = 20;
}
```

There can be separate information for:

Global Scope
x

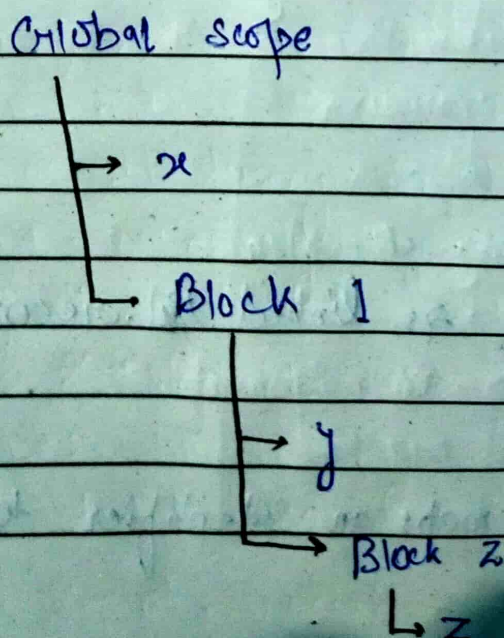
Block Scope
y

→ Nested Scope:

Consider:

```
int x;  
{  
    int y;  
    {  
        int z;  
    }  
}
```

The scope can be represented as:



For an identifier not found there, it searches the enclosing scope.

Current scope



Parent scope



Global scope

→ Symbol Table implement

A symbol table can be implemented using different data structures.

1. Linear List:

$x \rightarrow \text{int}$

y → float

$z \rightarrow \text{char}$

2. Linked List:

Symbol are stored as linked list record.

3 Hash terrible.

A hash function maps an identifier to a table location.

Identifiers



Hash function



Hash table

4. Tree:

A search tree can also be used to store identifiers.

→ Run-time Environment :-

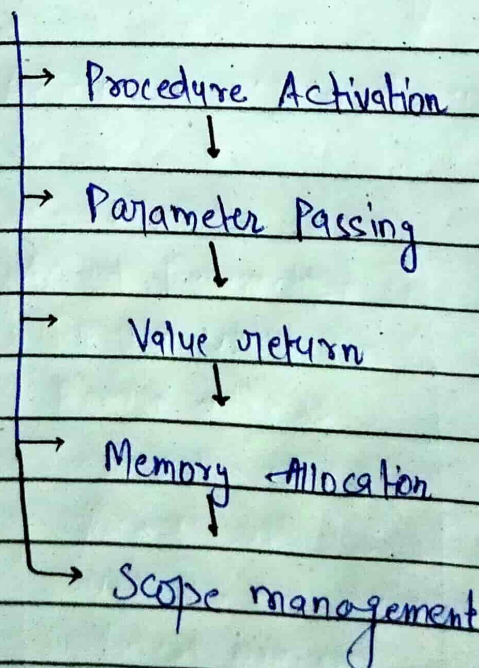
The run-time Environment (RTE) is the part of a compiler/runtime system that manages everything required while a program is executing.

It mainly deals with :

- Procedure/function call
- Passing parameters
- Returning values from functions
- Allocating and releasing memory
- Managing scope of variable

Flow :

Run Time Environment



→ Procedure Activation:~

A procedure activation means the execution of a procedure/function after it is called.

Eg:

main()



sum(10, 20)



return 30

when main() calls sum(), the system has to create an environment for sum() to execute.

This temporary execution environment is called an activation.

Eg;

```
function sum(a, b)
{
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

when we call:

sum(10, 20)

The run time system needs to Store :

$a = 10$

$b = 20$

$c = 30$

~~***~~ Return address

∴, a new activation is Created for Sum().

→ Activation Record :-

An activation record is a block of memory that stores information required for one execution of a procedure/ function.

It is also called Stack frame.

A typical Activation record contains :

Actual Parameters
Return value
Return address
Control link
Access link
Saved Machine Status
Local variable
Temporaries

Component

Purpose

Parameters

values passed to the function

Return value

value produced by the function

Return address

where execution should continue

Control link

Connects to caller's activation

Access link

Help access non-local variables

Local variables

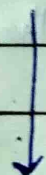
Variables declared inside funcⁿ

Temporaries

intermediate computation values

Eg;

main()



calls sum(10, 20)

Activation Record	
Parameters:	10, 20
Return address	
local :	C
Temporary values	

when sum() finishes, its activation record is normally removed from the runtime stack.

→ Parameter passing :

When a function is called, information may need to be transferred from the caller to the called function.

This is called parameter passing.

Eg :

sum(10, 20)

here,

10 and 20 are actual parameter.

a and b are formal parameter.

Actual parameter

formal parameter

10 →
20 →

a
b

(A) Call by Value :

In call by value, a copy of the actual parameter is passed to the function.

Eg ;

x = 10 ;

function change(a)
a = 20 ;

when $\text{change}(x)$ is called:

$x = 10$

↓ copy

$a = 10$

inside the function

$a = 20;$

but

$x = 10$

remains unchanged.

(B) Call by reference :-

In call by reference, the address/reference of the actual parameter is passed.

Eg:

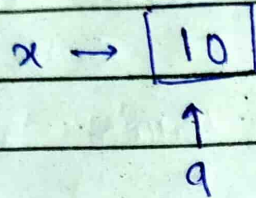
$x = 10$

function $\text{change}(*a)$

$*a = 20;$

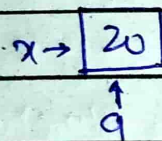
calling: $\text{change}(\&x)$

makes a refer to the same storage location as x .



if $a = 20$

then the same memory location changes:



$\therefore x = 20$

(c) Call by Name:

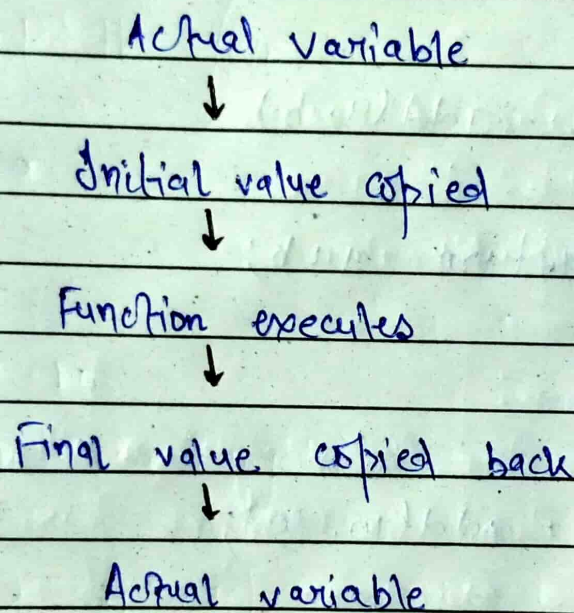
In call by name, the actual parameter is substituted/evaluated when it is used inside the procedure.

It behaves somewhat like textual substitution, but the exact semantics depends on the language.

It was historically used in language such as Algol 60.

D) Call by result:

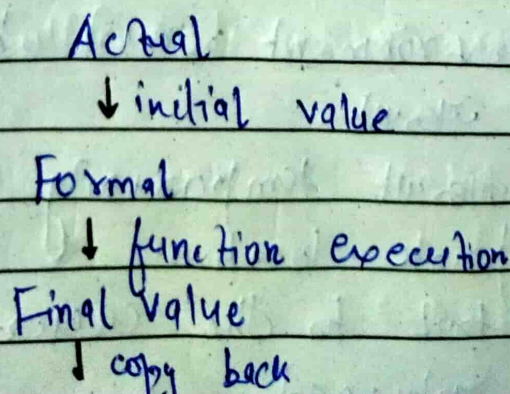
In call by result, the formal parameter gets a local value during execution, and its final value is copied back to the actual parameter when the function finishes.



E) Call by Value - Result:

It combines values and result.

1. Value is ~~the~~ copied from actual → formal.
2. Function work on the formal parameter.
3. Final value is copied formal → Actual.



→ Value Return :

A function may produce a ~~value~~ result that must be returned to its caller.

This is called value return.

Eg ;

```
function add(a, b)
{
    return a + b;
}
```

if :

```
result = add(10, 20);
```

then :

add()

↓

30

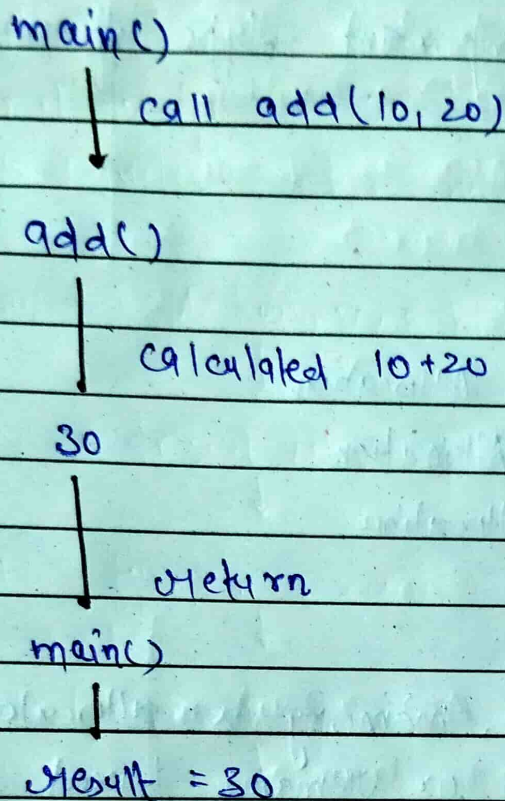
↓

result

The runtime environment must handle :

1. Calculating the result.
2. Storing the result temporarily or in an appropriate location.
3. Returning control to the caller.
4. Making the returned value available to the caller.

Example:



The return address stored in the activation record tells the system where execution should continue after the function returns.

→ Memory Allocation:

During program execution, memory is required for:

- Variable
- Function/procedures
- Parameters
- Arrays
- Objects
- Temporary value
- Dynamic data

The runtime environment manage this memory.

There are three important type of memory allocation:

Memory Allocation

- Static Allocation
- Stack Allocation
- Heap Allocation

(A) Static Allocation:

In Static allocation, memory is allocated before program execution, or remains fixed for the program's execution.

The memory location of an object generally remain fixed.

Eg;

global int x;

The storage for x can be allocated statically.

→ Characteristics :

- Allocation happens before / during program startup.
- Size is generally fixed.
- Suitable for global / static variables.
- Simple and fast.

→ Limitation :

It is not suitable when the number or lifetime of objects is determined dynamically.

(B) Stack Allocation :

Stack Allocation is commonly used for function / procedure ~~calls~~ calls.

Every function call creates an activation record on the stack.

eg;

main()



A()



B()

Stack in

	B() activation	
	A() activation	
	main()	

When B() finishes :

	A() activation	
	main()	

The activation record of B() is removed.

Advantages:

- Fast allocation/deallocation
- Naturally supports nested function calls.
- Supports recursion

Recursion example:

fact(3)



fact(2)



fact(1)

Each call gets its own activation record:

fact(1)
fact(2)
fact(3)

(c) Heap Allocation:

Heap allocation is used when memory must be allocated dynamically during program execution.

Eg;

new object

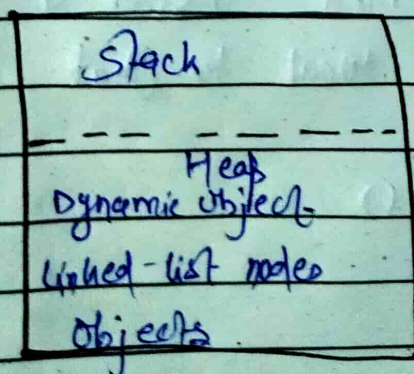
or dynamic data structure such as:

Linked list

Tree

Graph

Dynamic Array



→ Scope:

Scope defines the region of program in which a particular identifier can be accessed or used.

Eg;

```
int x = 10;
```

```
function A()  
{
```

```
    int y = 20;  
}
```

here:

- x may be accessible throughout is applicable scope.

- y is accessible only inside function A().

→ Type of Scope:

(A) Local Scope:

A variable declared inside a function/block is usually local to that region.

```
function test()  
{  
    int x = 10;  
}
```

x is accessible inside test() but not outside its scope.

(B) Global Scope:

A global ~~scope~~ variable is declared outside function and can be accessible from multiple parts of the program, subject to the language's rules.

```
int x = 10;
```

```
function A()  
{  
    use x;  
}
```

(C) Nested scope:

A block can be inside another block.

```
{
```

```
    int x = 10;
```

```
    {
```

```
        int y = 20;
```

```
        // x and y can be accessible here
```

```
    }
```

```
    // y is not accessible here
```

```
}
```

[Signature]